

Resumen Ejecutivo

El presente proyecto detalla el desarrollo de un sistema web integral orientado al control de asistencia y registro de retardos del personal, el cual se encuentra respaldado por una base de datos PostgreSQL alojada en Supabase y desplegado en la plataforma Vercel. Esta solución tecnológica surge como respuesta a la necesidad apremiante de reemplazar los registros manuales y la información dispersa por un sistema centralizado, altamente trazable y de fácil consulta.

El proceso de desarrollo se rigió metodológicamente bajo el enfoque del Desarrollo Guiado por Especificaciones (SDD, por sus siglas en inglés) en conjunto con el marco de trabajo ágil Scrum. En fases tempranas, se establecieron el planteamiento del problema, los objetivos, la justificación pertinente, así como las limitaciones y el alcance del sistema. Posteriormente, se efectuó el diseño del modelo de datos, estructurando los entregables mediante iteraciones (sprints) para culminar con la implementación de un Producto Mínimo Viable (MVP). Este MVP cuenta con módulos plenamente funcionales destinados a la gestión de empleados, control de turnos, registro de asistencia, administración de roles y emisión de reportes.

Como mecanismo fundamental para el aseguramiento de la calidad del software, se incorporaron pruebas unitarias orientadas a validar las reglas de negocio vinculadas al cálculo de tardanzas, tiempo laborado y consolidación de asistencias. Paralelamente, se desarrollaron pruebas de integración enfocadas en validar la correcta persistencia y consulta de datos mediante las rutas API (Application Programming Interface). Asimismo, se preparó un esquema SQL estructurado, un conjunto de datos semilla para simulaciones y una estrategia de despliegue completamente reproducible.

En síntesis, este proyecto no solo resuelve una problemática administrativa tangible, sino que también evidencia empíricamente el ciclo de vida completo del software, generando un respaldo documental y técnico riguroso.

Planteamiento del Problema

Planteamiento general

La gestión manual de la asistencia y los retardos de los empleados en entornos organizacionales provoca frecuentes errores de registro, dificulta el seguimiento riguroso de los horarios establecidos y, en consecuencia, reduce drásticamente la confiabilidad de los reportes administrativos utilizados para la toma de decisiones.

Problemas específicos

- El registro manual de las entradas y salidas de los empleados propicia la ocurrencia de omisiones humanas y la generación de datos inconsistentes.
- Existe una deficiencia en la trazabilidad de la información, lo cual imposibilita la identificación precisa de tardanzas, ausencias y otro tipo de observaciones laborales.
- El procesamiento y elaboración de reportes de asistencia demanda una inversión de tiempo excesiva al depender de procesos administrativos manuales y repetitivos.
- La información recolectada suele fragmentarse en diversas hojas de cálculo o formatos físicos, careciendo de un repositorio de control centralizado y seguro.

Objetivos

Objetivo general

Desarrollar e implementar un sistema web centralizado para el control de asistencia y retardos de empleados, el cual integre una base de datos relacional, mecanismos de pruebas automatizadas y despliegue en la nube, fundamentado en la aplicación estricta de las prácticas de Specification-Driven Development (SDD) y el marco ágil Scrum.

Objetivos específicos

- Analizar exhaustivamente las necesidades y requerimientos, tanto funcionales como no funcionales, inherentes al proceso de control de asistencia del personal.
- Diseñar la arquitectura del sistema y el modelo de datos relacional indispensable para el registro eficiente de empleados, configuración de horarios y captura de asistencias.
- Implementar una aplicación web interactiva que permita el registro preciso de entradas, salidas, cálculo de tardanzas y la consecuente generación de reportes.
- Incorporar rutinas de pruebas unitarias y de integración para validar la exactitud de las reglas de negocio y garantizar el flujo correcto de información entre los distintos módulos del sistema.
- Desplegar la aplicación en un entorno de producción en línea, documentando rigurosamente toda la evidencia técnica generada a lo largo del ciclo de vida del proyecto.

Justificación

El presente proyecto adquiere relevancia y justificación debido a su capacidad para automatizar un proceso administrativo crítico, recurrente y de alto impacto operacional en cualquier institución: el control de asistencia. La transición de registros manuales susceptibles a fallos hacia un sistema web respaldado por una base de datos centralizada conlleva una reducción significativa de errores humanos, una optimización sustancial en la trazabilidad de los datos y una mayor agilidad al consultar información histórica del personal.

Desde una perspectiva académica e ingenieril, el desarrollo de este sistema constituye un ejercicio integral que permite demostrar de manera práctica el ciclo de vida completo del desarrollo de software. Este abarca desde las fases iniciales de análisis y diseño arquitectónico, hasta la programación, implementación de pruebas automatizadas y despliegue final. Asimismo, proporciona evidencias concretas sobre la correcta aplicación de SDD y Scrum, garantizando que cada componente del sistema esté debidamente documentado mediante requisitos formales, historias de usuario, entregables incrementales por sprint y criterios de aceptación verificables.

Limitaciones y Alcance

Alcance del proyecto

El sistema de software desarrollado comprende los módulos para el registro y gestión de empleados, configuración de horarios (turnos), registro automatizado de entradas y salidas, cálculo de tardanzas y generación de reportes administrativos básicos. De igual forma, el alcance incluye la gestión de seguridad mediante roles de acceso diferenciados y la persistencia de la información en una base de datos centralizada, asegurando la integridad de los registros.

Limitaciones

- El sistema actual prescinde de integraciones con dispositivos de hardware especializado o sistemas de validación biométrica.
- La solución no abarca el procesamiento de nóminas ni la automatización de cálculos salariales o descuentos financieros.
- No se contemplan funcionalidades basadas en geolocalización o restricciones avanzadas vinculadas a direcciones IP por dispositivo.
- El análisis de los datos se limita a reportes estructurados, excluyendo en esta primera versión la implementación de analítica de datos compleja o algoritmos de inteligencia artificial.
- El producto final se enmarca en las características de un Producto Mínimo Viable (MVP), diseñado para ser demostrable, funcional y sustentable dentro de los tiempos estipulados para su desarrollo.

Marco Teórico

Ciclo de vida del software

El ciclo de vida del software comprende una serie de etapas estandarizadas que incluyen el análisis de requisitos, diseño de la arquitectura, fase de programación, ejecución de pruebas de calidad, despliegue y mantenimiento posterior. En el marco de este proyecto, cada fase ha sido formalmente documentada para evidenciar la evolución sistemática de la solución tecnológica, partiendo de una problemática planteada hasta la obtención de una aplicación web completamente funcional.

Specification-Driven Development (SDD)

El SDD se define como una metodología orientada a guiar el desarrollo de software a partir del establecimiento riguroso de especificaciones y diseños previos a la codificación. Bajo este paradigma, primeramente se redactan los requisitos funcionales, las reglas de negocio y los criterios de aceptación; posteriormente se estructura el diseño técnico y, finalmente, se procede con la implementación. Dicha implementación es constantemente verificada contra las especificaciones iniciales para garantizar la alineación total con las necesidades del sistema.

Marco de trabajo ágil Scrum

Scrum es un marco de trabajo colaborativo que fragmenta y organiza el ciclo de desarrollo en iteraciones cortas y manejables denominadas *sprints*. Para efectos de este proyecto, Scrum ha facilitado la segmentación del esfuerzo en entregables incrementales de alto valor, abarcando fases de análisis, diseño de base de datos, programación de la lógica, testing y despliegue continuo.

Aseguramiento de la calidad y pruebas

Las pruebas de software constituyen un pilar fundamental de la calidad. Las pruebas unitarias se enfocan en verificar el correcto funcionamiento de funciones matemáticas o reglas de negocio individuales de forma aislada. Por su parte, las pruebas de integración evalúan que los distintos módulos interactúen de forma esperada y coherente; por ejemplo, garantizando que, al registrar una asistencia desde la interfaz web, el dato sea correctamente transmitido, procesado y almacenado de forma persistente en la base de datos.

Despliegue en la nube

El despliegue representa la fase en la cual la aplicación se publica en un entorno accesible a través de internet, permitiendo la demostración en vivo del sistema. Para la infraestructura de este proyecto, se ha seleccionado a Vercel como plataforma de alojamiento optimizada para la

aplicación web construida en Next.js, en conjunto con Supabase para la administración en la nube de la base de datos relacional PostgreSQL.

Materiales y Métodos

Materiales y herramientas tecnológicas

- **Entorno de desarrollo integrado (IDE):** Visual Studio Code.
- **Control de versiones y repositorios:** Git y GitHub.
- **Desarrollo Frontend y Backend (Framework):** Next.js utilizando el lenguaje TypeScript.
- **Gestor de base de datos:** PostgreSQL administrado mediante la plataforma Supabase.
- **Plataforma de despliegue y alojamiento (Hosting):** Vercel.
- **Frameworks de pruebas (Testing):** Vitest y Testing Library.

Metodología de trabajo

1. Levantamiento, análisis y definición formal de los requisitos a partir de la formulación del problema organizacional.
2. Elaboración estructurada de historias de usuario acompañadas de sus respectivos criterios de aceptación.
3. Diseño normalizado del modelo relacional de la base de datos y estipulación de las reglas de negocio.
4. Implementación iterativa e incremental del código fuente del sistema (Backend y Frontend).
5. Elaboración de los *scripts* de pruebas automatizadas y ejecución del entorno de validación (QA).
6. Despliegue en producción de la solución tecnológica y compilación de la documentación formal de resultados.

Análisis y Diseño bajo SDD

Visión de producto

El sistema propuesto permite la centralización eficaz del control de asistencia y registro de retardos del personal a través de una aplicación web soportada por una base de datos segura. El administrador o usuario principal cuenta con las facultades para registrar colaboradores, configurar turnos operativos, capturar los marcajes de asistencia y consultar reportes históricos; eliminando por completo la dependencia hacia herramientas ofimáticas manuales y descentralizadas.

Requisitos funcionales

- **RF01:** El sistema debe registrar nuevos empleados solicitando código único, nombre completo, correo electrónico y estado laboral (activo/inactivo).
- **RF02:** El sistema debe permitir la creación de turnos laborales definiendo la hora de inicio, hora de finalización y el tiempo de tolerancia expresado en minutos.
- **RF03:** El sistema debe posibilitar el registro de las horas de entrada y salida asociadas a cada empleado por fecha calendario.
- **RF04:** El sistema debe calcular automáticamente los minutos de tardanza cruzando la hora de ingreso con el turno asignado al empleado.
- **RF05:** El sistema debe exponer una interfaz para consultar el historial detallado de asistencia por colaborador.
- **RF06:** El sistema debe generar reportes analíticos básicos filtrados por un rango de fechas determinado.
- **RF07:** El sistema debe gestionar el control de acceso basándose en roles administrativos definidos (ej. Administrador y Supervisor).

Requisitos no funcionales

- **RNF01:** La aplicación deberá ser plenamente accesible desde cualquier navegador web moderno.
- **RNF02:** La totalidad de la información estructurada deberá ser persistida utilizando el motor de bases de datos PostgreSQL.
- **RNF03:** La lógica algorítmica crítica (cálculo de tiempos) deberá poseer una cobertura de pruebas unitarias.
- **RNF04:** Los módulos centrales de registro deberán validar la integridad del flujo de datos mediante la ejecución de pruebas de integración.
- **RNF05:** La aplicación web deberá estar optimizada para ser desplegada en un entorno de alojamiento público en la nube.

Historias de usuario

- **HU01:** Como administrador, requiero registrar los datos de los empleados para mantener un directorio unificado y control central sobre el personal.
- **HU02:** Como administrador, requiero configurar y definir turnos de trabajo para posibilitar la comparación automatizada entre la hora programada y la hora real de llegada.
- **HU03:** Como encargado de asistencia, requiero una interfaz rápida para asentar las entradas y salidas, con el fin de consolidar la asistencia diaria del personal.
- **HU04:** Como supervisor operativo, requiero visualizar las tardanzas acumuladas para la detección temprana de incumplimientos de horario.
- **HU05:** Como supervisor, requiero consultar informes consolidados por fechas para auditar el comportamiento histórico de la plantilla.

Criterios de aceptación

- **CA01:** El aplicativo almacena de manera exitosa la información de empleados y turnos directamente en la tabla correspondiente de la base de datos.
- **CA02:** La lógica de negocio bloquea y no permite la duplicidad de registros de entrada para un mismo empleado en una misma fecha.
- **CA03:** El algoritmo del sistema calcula correctamente la cantidad de minutos de tardanza al detectar que la hora de marcación posterior excede la tolerancia del turno.
- **CA04:** Las pantallas de historial y reportes exponen cifras congruentes con los datos puros almacenados en la base de datos.
- **CA05:** Las reglas críticas de cálculo superan sin errores el *pipeline* de pruebas automatizadas.

Trazabilidad y arquitectura inicial

El análisis de requisitos fue traducido a componentes estructurales mediante la siguiente matriz de trazabilidad de diseño inicial:

- La entidad `employees` satisface el RF01.
- La entidad `shifts` satisface el RF02.
- La entidad `attendance_records` satisface el RF03.
- La lógica de negocio `calculateTardinessMinutes` satisface el RF04.
- Las consultas `SQL/API` por `employee_id` satisfacen el RF05.
- La arquitectura de componentes de Interfaz de Usuario (UI) satisface el RF06.

Suposiciones del diseño del sistema

- El formato horario de referencia adoptado será exclusivamente el de 24 horas.

- La tardanza será procesada matemáticamente en números enteros (minutos).
- Cada colaborador dispondrá, como máximo, de un ciclo único de asistencia (una entrada y una salida) por cada fecha natural.

Gestión con Scrum

Product Backlog (Pila del Producto)

1. Módulo para el registro y gestión de empleados.
2. Módulo para el registro y parametrización de horarios laborales.
3. Kiosko virtual para el registro de marcaciones (entradas y salidas).
4. Motor lógico para el cálculo y clasificación de tardanzas.
5. Panel analítico para la generación de reportes de asistencia.
6. Preparación del entorno y despliegue final en la nube.

Planificación de Sprints

La distribución del trabajo se organizó en las siguientes iteraciones:

- **Sprint 1:** Análisis profundo de la problemática, delimitación del alcance y redacción formal de requisitos técnicos.
- **Sprint 2:** Diseño arquitectónico de la base de datos, creación del modelo relacional y estructuración del repositorio.
- **Sprint 3:** Programación e implementación iterativa de los módulos de registro, autenticación y panel de control.
- **Sprint 4:** Desarrollo de la suite de pruebas unitarias/integración, corrección de defectos técnicos (bug-fixing) y despliegue del sistema a producción.

Entregables por iteración

Al finalizar cada Sprint, se obtuvieron los siguientes artefactos funcionales y documentales:

- Documentos técnicos y diagramas arquitectónicos formateados en Markdown.
- Modelado de datos implementado en PostgreSQL.
- Código fuente ejecutable correspondiente al Producto Mínimo Viable (MVP).
- Evidencia de ejecución satisfactoria de las rutinas de aseguramiento de calidad (QA).
- Enlace público con la evidencia de la aplicación desplegada.

Diseño de Base de Datos

El ecosistema de datos se encuentra estructurado bajo el modelo relacional y es hospedado mediante PostgreSQL (Supabase). La arquitectura de información contempla fundamentalmente las siguientes cuatro tablas centrales:

1. Tabla: employees

Repositorio para el almacenamiento de la información del personal adscrito a la organización.

- `id` (uuid, Clave Primaria)
- `employee_code` (text, Restricción Única)
- `full_name` (text)
- `email` (text)
- `active` (boolean)
- `created_at` (timestampz)

2. Tabla: attendance_records

Entidad transaccional encargada de persistir el registro temporal diario de cada colaborador.

- `id` (uuid, Clave Primaria)
- `employee_id` (uuid, Clave Foránea referenciando a `employees.id`)
- `work_date` (text)
- `clock_in` (text)
- `clock_out` (text, opcional)
- `tardiness_minutes` (integer)
- `notes` (text, opcional)
- `shift_id` (text, Clave Foránea referenciando a `shifts.id`)
- `created_at` (timestampz)

3. Tabla: shifts

Catálogo que define y parametriza los horarios laborales operativos y los márgenes de tolerancia permitidos.

- `id` (text, Clave Primaria)
- `name` (text)
- `start_time` (text)
- `end_time` (text)
- `tolerance_minutes` (integer)

- `created_at` (timestampz)

4. Tabla: roles

Entidad para definir la estructura de permisos y accesos autorizados al sistema (ej. Administrador, Supervisor, Empleado).

- `id` (text, Clave Primaria)
- `name` (text)
- `created_at` (timestampz)

Relaciones Principales

- Se establece una relación de **Uno a Muchos (1:N)** entre la entidad `employees` y `attendance_records`. Un único empleado posee la capacidad de generar múltiples registros históricos de asistencia (identificados mediante `employee_id`).
- Se establece una relación de **Uno a Muchos (1:N)** entre la entidad `shifts` y `attendance_records`. Un turno de trabajo específico se aplica simultáneamente a los registros de asistencia de múltiples colaboradores (vinculados a través de `shift_id`).

Reglas de integridad y negocio

- Es mandatorio que cada empleado cuente con un código identificador alfanumérico irrepetible (Unique constraint).
- Un registro transaccional de asistencia no podrá duplicarse en la base de datos para una misma llave compuesta por fecha (`work_date`) y empleado (`employee_id`).
- El cálculo de tardanza se ejecuta en el servidor (backend) a nivel de la capa lógica antes de efectuar la inserción en la base de datos, garantizando así la pureza e inmutabilidad de la información histórica.
- Las vistas y consultas de reportes realizan cálculos agrupados estrictamente sobre la información persistida, inhibiendo manipulaciones o sobreescrituras manuales de resultados.

Implementación

La arquitectura del sistema fue construida bajo un modelo cliente-servidor de alta responsividad (Single Page Application y Server-Side Rendering) utilizando el ecosistema tecnológico de Next.js (versión 15).

El entorno Frontend (Capa de Presentación) fue edificado utilizando React y estilizado con componentes reutilizables a través de Tailwind CSS, priorizando una interfaz de usuario limpia e intuitiva en formato *Dark Mode*. Por su parte, el Backend (Capa de Negocio) aprovecha las capacidades de Next.js API Routes bajo el entorno Node.js, actuando como un intermediario seguro para la gestión, sanitización y transferencia de datos hacia la base de datos PostgreSQL alojada en Supabase.

Para asegurar la rigurosidad en la validación de la información de entrada, se integró la librería Zod (Schema Validation), la cual actúa como un filtro estricto que rechaza cargas útiles (payloads) que no cumplan con el contrato de la API.

Pruebas (Unitarias e Integración)

Como mecanismo indispensable para el cumplimiento de los más altos estándares de calidad, se implementó una suite de validación automatizada mediante el framework **Vitest**.

Pruebas unitarias ejecutadas

Las pruebas unitarias fueron diseñadas para comprobar matemáticamente la validez de los algoritmos de negocio aislados:

- Cálculo automatizado de los minutos de tardanza en función del umbral de tolerancia del turno.
- Algoritmo de clasificación y discernimiento entre asistencia puntual y asistencia con retardo.
- Consolidación sumatoria del tiempo total trabajado en minutos enteros.
- Validación lógica de casos límite (Edge Cases), como intentos de registro en los que la hora de salida precede ilógicamente a la hora de entrada.
- Comportamiento esperado de los resúmenes estadísticos cuando el empleado carece de un historial previo.

Pruebas de integración

Estas validaciones buscaron ratificar el comportamiento sistémico al momento en que diversos componentes de software convergen:

- Confirmación de que el registro de asistencia atraviesa el protocolo API y culmina en una persistencia exitosa dentro de la base de datos de pruebas.
- Verificación del *endpoint* de historial asegurando que los datos devueltos coincidan con la identidad del empleado solicitado.
- Respuestas de códigos de estado correctas (HTTP 200 OK frente a HTTP 400 Bad Request) al enviar modelos de datos válidos e inválidos a las rutas de empleados y turnos.

Resultados de la ejecución técnica

El *pipeline* de validación demostró una eficacia del 100%. Todos los casos de prueba diseñados para evaluar los componentes críticos, reportes y la integración nativa de la API fueron procesados exitosamente, logrando así descartar anomalías de cálculo previo al despliegue en producción.

Despliegue

Estrategia y flujo operativo

Para garantizar un entorno de alta disponibilidad durante la demostración académica del producto, se llevó a cabo una estrategia de Integración y Despliegue Continuo (CI/CD).

1. La base de datos, en primera instancia, fue inicializada en el servicio administrado Supabase, en el que se ejecutó el archivo estructurado y posteriormente el script de inicialización para proveer al sistema de un escenario base de información controlada.
2. El código fuente versionado en la rama principal del repositorio de GitHub fue vinculado automáticamente con la plataforma Vercel.
3. Se aprovisionaron de forma segura las variables de entorno de producción (credenciales criptográficas hacia Supabase) en el panel de Vercel.
4. El proceso de compilación generó el despliegue de una URL pública plenamente accesible para validar las funciones del aplicativo desde dispositivos externos.

Resultados

Consecución de resultados

- Se obtuvo exitosamente un sistema de información capaz de orquestar flujos completos de registro de empleados, asociación de turnos, captura de asistencias y discriminación de roles desde una moderna interfaz web centralizada.
- La base de datos fue estructurada acorde al diseño relacional propuesto y es consumida a través de rutas API REST seguras.
- Las rutinas de pruebas automatizadas constataron que las reglas algorítmicas de detección de tardanzas operan sin margen de error.
- Se consolidó un entorno técnico estable implementando Supabase y Vercel, dotando al sistema de capacidades de operación real.

Indicadores y métricas de éxito

El reemplazo del procesamiento manual y descentralizado por la solución de asistencia refleja una mitigación directa del error humano durante las capturas diarias. La correcta relación foránea entre tablas (empleados, turnos y asistencias) asegura una trazabilidad impecable de la información. Además, los cuadros de mando de la aplicación demostraron su viabilidad para generar reportes numéricos instantáneos y consistentes, basados única y exclusivamente en registros persistidos matemáticamente inmutables.

Este conjunto de logros materializa los objetivos planteados al inicio y ofrece evidencia documentada de las mejores prácticas de la ingeniería de software como el diseño basado en especificaciones (SDD) y metodologías ágiles.

Conclusiones y Recomendaciones

Conclusiones

El desarrollo del proyecto valida que la implementación de un ciclo de vida completo del software—contemplando fases de diseño documental formal, programación modular, pruebas de control de calidad y despliegue automatizado—garantiza la construcción de un producto de software de alto impacto y fiabilidad. Queda probada la pertinencia de la combinación del marco ágil Scrum con el enfoque SDD, puesto que enlazar de manera anticipada cada requisito del cliente final con la arquitectura técnica pertinente, previno retrabajos significativos.

Desde un prisma formativo, la dimensión técnica del sistema probó ser ideal como caso de estudio académico; su alcance fue acotado de manera inteligente para permitir la demostración exhaustiva del ciclo ingenieril sin depender de arquitecturas físicas restrictivas o configuraciones biométricas. Todo esto consolida una solución web que favorece notablemente la trazabilidad laboral, optimiza el consumo de tiempo administrativo y brinda una sólida defensa técnica para el presente ejercicio final.

Recomendaciones

- Se sugiere sostener a futuro la disciplina de trazabilidad metodológica para futuras actualizaciones, conservando el cruce de datos entre las historias de usuario y los nuevos módulos a codificar.
- Para fases productivas de mayor escala, se aconseja robustecer la matriz de pruebas de integración abordando rutinas *end-to-end* (E2E) con la finalidad de automatizar interacciones complejas desde el navegador.
- Resultaría beneficioso explorar integraciones mediante la adición de servicios web que ofrezcan validación perimetral por medio de geolocalización o lectura de dispositivos biométricos a través de la red local.
- Asegurar de manera proactiva copias periódicas de la estructura de base de datos implementando un sistema automatizado de respaldos sobre las entidades críticas.

Referencias

Next.js. (2024). *Next.js Documentation*. Vercel Inc. <https://nextjs.org/docs>

Schwaber, K. & Sutherland, J. (2020). *The Scrum Guide. The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org.

Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.

Supabase. (2024). *Supabase Documentation*. Supabase Inc. <https://supabase.com/docs>

Testing Library. (2024). *Testing Library Official Documentation*. <https://testing-library.com/docs>

Vitest. (2024). *Vitest Official Guide*. <https://vitest.dev>

Anexos

A continuación, se referencian los insumos visuales y documentales complementarios al desarrollo tecnológico del sistema:

- Matriz formal de trazabilidad que cruza los Requisitos Funcionales, Historias de Usuario, tablas relacionales en base de datos y la cobertura de pruebas de software.
- Bitácora e historial de cambios registrada en el repositorio remoto GitHub como evidencia fehaciente del esfuerzo iterativo y de integración continua.
- Capturas de pantalla comprobatorias de las interfaces de Kiosko de empleado, panel gerencial de administración, y del registro seguro de autenticación.
- Capturas de la terminal comprobando la ejecución de los casos de prueba diseñados bajo el entorno automatizado.
- Respaldo impreso o digital del diseño de la base de datos (Modelo Entidad-Relación) y de los esquemas necesarios para la inicialización.